Pontificia Universidad Católica de Chile
Escuela de Ingeniería
IIC3253 – Criptografía y Seguridad Computacional
Primer Semestre del 2026

Ayudantía 08

Hash-Based Message Authentication Code HMAC

Ayudante: Manuel Villablanca – `manuel.villablanca@uc.cl`

Problema 1

¿Por qué en la construcción de Merkle-Damgård $Pad(m)$ incluye el largo del mensaje? ¿Qué propiedad necesaria se rompería en caso contrario? Dé un ejemplo concreto para ilustrar qué pasa en el caso de que no se incluya el largo del mensaje.

Solución

Si $|m|$ no es el múltiplo del largo del bloque, entonces:

$$\text{Pad}(m) = m \| p_m$$

Ahora, si hasheamos directamente un m' tal que

$$m' = m \| p_m = \text{Pad}(m)$$

Entonces encontramos una colisión, ya que m' y m tienen el mismo Hash.
Ejemplo concreto:

Consideremos un escenario simple con un tamaño de bloque de 8 bits y un mensaje original:

$$m_1 = A \quad \text{(que en ASCII es 01000001)}$$

Supongamos que usamos un padding simple sin incluir la longitud:

$$\text{Pad}(m_1) = 01000001 \| 10000000 = 0100000110000000$$

Donde añadimos el bit "1" seguido de ceros para completar un múltiplo del tamaño de bloque. Ahora, un atacante podría crear un mensaje malicioso $m_3 = m_1 \| 10000000$, es decir, "A" seguido del byte de padding que usamos para m_1.
El problema es que sin incluir la longitud original en el padding:

$$h(m_1) = h(\text{Pad}(m_1)) = h(01000001 \| 10000000)$$

Y para m_3:

$$h(m_3) = h(01000001 \| 10000000) = h(\text{Pad}(m_1))$$

Por lo tanto, $h(m_3) = h(m_1)$, ¡lo que es una colisión!
En cambio, con el padding adecuado que incluye la longitud:

- Para m_1:
$$\text{Pad}(m_1) = 01000001 \| 10000000 \| 00001000$$
donde 00001000 es la representación binaria de 8, la longitud original en bits.
- Para m_3:
$$\text{Pad}(m_3) = 01000001 \| 10000000 \| 10000000 \| 00010000$$
donde 00010000 es la representación binaria de 16, la longitud original en bits.

Ahora $h(m_1) \neq h(m_3)$ porque sus representaciones con padding son diferentes, evitando la colisión y preservando la seguridad de la función hash.

Problema 2

JSON Web Token (JWT) es un estándar que permite transmitir información de forma segura. Un JWT tiene la forma:

Header.Payload.Signature

Estructura

- **Header:** Contiene el tipo de token y el algoritmo de firma utilizado.

```
Header = {"alg": "HS256", "typ": "JWT"}
```

- **Payload:** Contiene los "claims" o afirmaciones sobre una entidad y datos adicionales.

```
Payload = {
    "sub": "1234567890",
    "name": "Usuario Ejemplo",
    "admin": true
}
```

- **Signature:** Se calcula utilizando HMAC sobre el header y payload codificados en base64.

La firma se calcula de la siguiente manera:

```
Signature = HMAC-SHA256(base64UrlEncode(Header) + '.'
+ base64UrlEncode(Payload), secret)
```

Un ejemplo de JWT completo sería:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IlVzdWFyaW8gRWplbXBsbyIsImFkbWluIjp0cnVlfQ.
rDZ6zA5T823vxlFBwXXDhxPE-GSYxNYnMl5jP1PL1JQ
```


El receptor del token puede verificar su autenticidad recalculando la firma con la clave secreta compartida.

Afirmaciones

En base a lo anterior, responde si las siguientes afirmaciones son verdaderas o falsas, justificando tu decisión:

1. Si la clave secreta utilizada en HMAC para JWT es revelada, la seguridad del sistema permanece intacta.
2. Un atacante que intercepta un JWT válido puede modificar el payload para cambiar la identidad del usuario o sus privilegios sin invalidar la firma.
3. HMAC proporciona las mismas garantías de seguridad que un simple hash del mensaje cuando se usa en JWT.
4. Utilizar $MACk(M) = h(k||M)$ en lugar de HMAC para generar la firma del JWT es seguro contra ataques de extensión.

Solución

1. **Falso.** La clave secreta es fundamental para la seguridad de JWT porque permite calcular y verificar la firma. Si la clave secreta se revela, un atacante podría crear tokens fraudulentos con cualquier contenido y firmarlos correctamente, suplantando así cualquier identidad o privilegio.
2. **Falso.** Un atacante no puede modificar el payload sin invalidar la firma. Cualquier cambio en el header o payload resultaría en una firma diferente al recalcular el HMAC. Como el atacante no posee la clave secreta, no puede generar una firma válida para un payload modificado.
3. **Falso.** HMAC proporciona tanto integridad como autenticación, mientras que un simple hash solo garantiza integridad. Además, HMAC es resistente a los ataques de extensión de longitud y otros ataques que afectan a esquemas como $h(k\|M)$ o $h(M\|k)$.

4. **Falso.** Utilizar

$$MAC_k(M) = h(k\|M)$$

es vulnerable a ataques de extensión. Un atacante que tiene un token válido podría extender el payload añadiendo información adicional sin conocer la clave, aprovechando la estructura iterativa de las funciones hash basadas en Merkle-Damgård.

Problema 3

1. Demuestre que definir $MAC_k(M) = h(k\|M)$ resulta en un MAC inseguro. Es decir, demuestre que dado un par texto/MAC válido (M, H), se puede construir eficientemente otro par texto/MAC válido (M', H') sin conocer la clave k. Asuma por simplicidad que la longitud de la clave es igual a la longitud del bloque del mensaje $|k| = |M_i|$.

   Pista: La función de compresión h' es, por supuesto, conocida, es decir, dadas las dos entradas cualquiera puede calcular su salida.

2. Demuestre que añadir la clave secreta k al final, es decir, definir $MAC_k(M) = h(M\|k)$ resulta en un MAC inseguro. Recuerde que, por definición, la propiedad de ser resistente a colisiones para un MAC significa que encontrar dos mensajes $x \neq x'$ tales que:

$$MAC(x, k) = MAC(x', k),$$

   implica el esfuerzo computacional de 2^n operaciones, donde n es el tamaño del MAC (y del hash). Describa un ataque (basado en la estructura Merkle-Damgård anterior) que utilice menos de 2^n operaciones para crear una falsificación MAC, un valor MAC legítimo para algún mensaje x' sin revelar la clave k.

3. Seria seguro definir $MAC_k(M) = h(k||h(M))$? Argumente.

Solución

a) Un adversario crea el mensaje $M' = x\|y$ por lo que necesita calcular $MAC_k(k\|x\|y)$. Pero la estructura iterativa del esquema Merkle-Damgård implica que:

$$MAC_k(k\|x\|y) = h'(MAC_k(k\|x), y)$$

   Por lo tanto, dado el valor $MAC_k(k\|x)$, el adversario simplemente evalúa la función de compresión públicamente conocida.

b) Aquí el procesamiento del mensaje no depende de la clave hasta el último bloque, por lo que podemos utilizar un *birthday attack*. Es decir, creamos $2^{n/2}$ mensajes diferentes:

$$M^1, \ldots, M^{2^{n/2}}$$

de la misma longitud (digamos que todos los mensajes tienen t bloques) y calculamos los valores hash $h(M^i)$. Entonces, la paradoja del cumpleaños implica que hay $M^i \neq M^j$ tales que

$$h(M^i) = h(M^j)$$

para algún $1 \leq i < j \leq 2^{n/2}$. Entonces:

$$MAC_k(M^i \| k) = h'(h(M^i), k) = h'(h(M^j), k) = MAC_k(M^j \| k)$$

El adversario puede simplemente obtener el valor MAC en el mensaje M^i, y luego producir un par texto-MAC correcto $(M^j, MAC_k(M^j \| k))$.

c) Siguiendo la misma logica de b, un adversario puede encontrar una colision en la funcion de hash con $2^{\frac{n}{2}}$ operaciones, lo que luego también genera una colisión en el MAC planteado.